

17. Abstraction & Polymorphism in Depth

▼ What is Abstraction ?

Abstraction is the practice of exposing functionalities, while hiding internal implementation details. It is the process of focusing on what something does, rather than how it does that.

Humans have evolved to work through abstraction. To drive a car, we just need to know how to operate it without knowing how it's working underneath.

▼ How many levels of abstraction does java offer?

Two levels

- Low Level (hiding implementation details)
- High Level (Separate the WHAT from the HOW)

▼ How does java implement low level abstraction ?

Through the use of classes. Encapsulation principles states that the method and behaviours of an object should be bound together. It is through this bounding, i.e. class, that we are able to expose certain behaviours.

▼ How does java implement high level abstraction?

Using **Interfaces** and **Abstract Classes**

High level abstraction separates the **what** from the **how**. However, this **how** and **what** are **tightly coupled** with each other, as every **what** has a specific **how**

Let's understand this through the following example

```
class FuelCar {  
    void drive(){  
        // implementation  
    }  
}
```

```

}

class ElectricCar {
    void drive() {
        // implementation
    }
}

```

Both the classes share the same **what**, but different **how's** i.e. internal implementation.

```

class Main {

    public static void main(String[] args) {
        FuelCar fc = new FuelCar();
        fc.drive();

        ElectricCar ec = new ElectricCar();
        ec.drive();
    }
}

```

The **reference variable** and the implementation(through **object**) are tightly coupled with each other

We might have a different type of Car tomorrow, which again has the same **what**, but a different **how** underneath. As a result, the user would then have to again create a different variable each time to achieve the same **what**

We basically want to achieve this

```

Car c = new ElectricCar();

// Later we might want a different implementation
c = new FuelCar();

```

Here, the variable and the object are not tightly coupled with each other. We know that this is possible through Inheritance, where the child class' object can be referenced via the parent reference

However we can go even beyond inheritance level and use **interfaces** instead

▼ What are abstract classes?

Abstract Class are java's way of implementing high level abstraction, by loosely coupling the implementing object and the reference variable.

We can declare methods without giving it's actual implementation there. Any child class extending must implement these methods or declare them abstract as it is

```
abstract public class Car {  
  
    void start() {  
        System.out.println("The car has started");  
    }  
  
    void honk() {  
        System.out.println("Beep Beep!");  
    }  
  
    abstract void accelerate();  
    abstract void brake();  
}
```

▼ Why can we not use the new keyword with Abstract classes?

Abstract classes can not be initiated using the new keyword, as it may have some abstract methods without their implementation present.

So, at run time the compiler won't be able to invoke `obj.someAbstractMethod()`

▼ What is an Interface?

Interface is a contract, which any implementing class must follow

▼ What are default methods in an Interface?

default methods are pre-defined methods inside the interface

They are properties of the object, and not the Interface, hence can not be invoked using `InterfaceExample.someDefaultMethod()`

The implementing class may or may not override it

▼ What are the rules of an interface?

→ **default methods** can not be private (otherwise how will any implementing object access/override?)

→ **abstract methods** are public by default in Interfaces. They can only have private static methods implemented internally

→ abstract and default methods are properties of the implementing classes' object and not the Interface

→ Any **fields** declared inside Interfaces are by default and can only be **public**, **static** and **final**

→ Static fields and methods are properties of the Interface and not the implementing class

→ Hence, object variables of classes implementing an interface, can not invoke neither static methods nor fields defined on the interface. They must be accessed using `InterfaceExample.someStaticMethod()` or `InterfaceExample.someStaticField`

```
public interface Car {  
  
    String color = "Black";  
  
    static void greet() {  
        System.out.println("Car says Hi!");  
    }  
}
```

```
}  
}
```

```
public class ElectricCar implements Car {  
    // ...  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        Car c = new ElectricCar();  
        c.greet(); // not allowed, will throw erro  
r  
        c.color; // not allowed, same  
    }  
}
```

▼ What is method hiding?

When a **subclass** declares a **static method** with the same method signature as that of the static method of parent, at run time instead of overriding the parent's method, it gets hidden or shadowed

```
class Parent {  
  
    int age = 43;  
  
    static void greet() {  
        System.out.println("Parent says Hi!");  
    }  
  
}
```

```
public class Child extends Parent{

    int age = 24;

    static void greet() {
        System.out.println("Child says Hi!");
    }

}
```

```
public class Main {

    public static void main(String[] args) {
        Parent p1 = new Parent();
        p1.greet(); // Parent's greet invoked

        Child c1 = new Child();
        c1.greet(); // Child's greet invoked

        Parent p = new Child();
        p.greet(); // Parent's greet invoked, Child's greet gets shadowed
    }

}
```

▼ What is field hiding?

Similar to method hiding, `variable.fieldName` fetches the field of the declared type of the variable and not the initiated type

```
Child c = new Child();
System.out.println(c.age); // print's Child's age: 24

Parent p = new Child();
System.out.println(p.age); // print's Parent's age: 43
```

▼ Are Interfaces a type of Inheritance?

Kind of, as because of high level abstraction and loose coupling of variable type and object type, they can be classified inheritance of type

They provide code re-usability at the design and composition level. But they do not provide direct code re-usability through inherited methods (except default methods)

▼ Interfaces vs Abstract Classes

Interfaces represent what they can do, and provide a contract that any implementing class must follow.

Interfaces can offer pure abstraction, without the need to implement anything internally.

Interfaces have no relation with the object's created.

Abstract classes helps to group families of similar objects.

▼ When should we use either Interface or an Abstract Class?

We should use an Abstract class only when we want to establish an "Is-a" relationship between closely related classes and want to share internal state or non-public code

→ If we are only looking establish a contract, then we should use Interface

▼ What is a good naming convention for Interfaces?

→ Interfaces represent *what it can do*, hence their naming convention is as such. For example, Runnable, Comparable, Serializable, Flyable etc. The names are in a **verb form** hinting towards what the Interface can do for any class looking to implement it

▼ Abstraction vs Encapsulation?

Encapsulation aims to bind the data members and functionalities, which it achieves through class. It also aims to provide security by protecting the internal members from outside access using Access Modifiers.

Abstraction aims only to expose certain functionalities or behaviour by hiding the internal implementation. You may inspect it, but even without it, you can

use it perfectly fine. It is not related with security and protection

▼ What is Polymorphism?

The name is made up of two words Poly (Many) and Morphism(forms/behavior)

It is the property of an entity to display different behaviours for the same functionality

▼ What is compile time polymorphism?

Method overloading

▼ What is runtime polymorphism?

Method Overriding

It is only possible when there is an inheritance relationship or when a class implements an interface

▼ Why is method overriding called run time polymorphism?

Because it is decided at run time which implementation to call based on the reference variable of the object.

Why at run time? Because get created only during the run time itself

→ Polymorphism can happen after the declaration and instantiation of an object

→ At run time, `variable.method()` depends on the reference type of the instance variable, unless that method is static